

Multi-Tenant RAG System

System Architecture, Security Isolation, and Implementation Specification • **Author:** Muhammad Ismail Tanoli

1. Executive Summary & Design Philosophy

Standard multi-tenant Retrieval-Augmented Generation (RAG) architectures frequently ingest documents from multiple organizations into a unified vector index, segregating records using soft metadata tags (e.g. `tenant_id: acme`). In production environments, this poses significant security and compliance vulnerabilities: improper filtering, vector index leakage, or adversarial prompt injections can expose confidential corporate data across tenants. This project implements **strict physical multi-tenancy** with separate vector database instances per organization, dual-factor authentication (Company credentials + Google OAuth 2.0 verification), persistent identity-keyed rate limiting, and an optimized small-talk filter.

2. Technical Stack & Component Specification

Subsystem	Technology / Model	Key Functional Role
Frontend	React 18, TypeScript, Vite, Tailwind CSS	Responsive dark glassmorphism UI, real-time quota gauges, 500-char input limiter.
Backend API	FastAPI (Python 3.13), Pydantic v2, Uvicorn	Asynchronous REST service, route-level authorization, custom CORS and security handlers.
Vector Storage	ChromaDB (Isolated physical collections)	Separate disk persistence per tenant (<code>tenant_acme</code> vs <code>tenant_globex</code>).
Embeddings	all-MiniLM-L6-v2 (Sentence-Transformers)	384-dimensional dense semantic vectors with Cosine Similarity index.
LLM Inference	Groq (LLaMA 3.3 70B Versatile)	Sub-second grounded response synthesis constrained strictly to tenant context.
Identity & Auth	Google OAuth 2.0 + PyJWT + Passlib (bcrypt)	Dual-factor authentication, cryptographic verification of Google ID token claims.
Rate Limiting	SlowAPI + Custom SQLite Storage	Persistent sliding-window counters keyed by permanent user Google ID (<code>google_sub</code>).

3. End-to-End System Flow of Control

The system enforces a deterministic, multi-stage pipeline for every incoming user request:

- **Phase 1 — Tenant Selection & Credentials Check:** The user selects Acme Corp or Globex Corporation and enters the company password. Passwords are cryptographically validated against pre-hashed bcrypt secrets stored in `tenants.yaml`.
- **Phase 2 — Google Identity Verification:** The browser completes Google OAuth 2.0. The backend verifies the Google ID token signature against Google's public JWKS certificates and extracts the immutable subject identifier (`google_sub`).
- **Phase 3 — Scoped JWT Session Issuance:** FastAPI signs a JWT access token embedding `tenant_id`, `google_sub`, and email. The token expires in 60 minutes and is maintained in client-side memory state.
- **Phase 4 — Fast Tenant Switching:** Users moving between organizations supply the new company password alongside their active Google token. The backend verifies tenant access and reissues a scoped token without redundant Google logins.
- **Phase 5 — Guardrails & Retrieval:** Inbound queries pass through the 500-character validator, small-talk filter (bypassing vector search for greetings), and identity-keyed SQLite rate limiter before querying the tenant's isolated ChromaDB database.
- **Phase 6 — Grounded Synthesis:** Retrieved text chunks populate a hardened system prompt. Groq LLaMA 3.3 synthesizes a concise, factual answer with raw file paths and leaky citations completely stripped.

4. Multi-Tenant Physical Isolation Architecture

To guarantee zero cross-tenant contamination, the application implements physical data partitioning rather than virtual logical tagging:

Physical Directory Partitioning:

- Acme Corp Collection: `data/chroma/tenant_acme` (indexed from Acme Corp Employee Handbook.pdf)
- Globex Corporation Collection: `data/chroma/tenant_globex` (indexed from Globex Corporation Employee Handbook.pdf)
- Separate Persistent Clients: Separate ChromaDB client connections prevent cross-collection querying in memory.

Cryptographic Route Enforcement: At the API layer (POST /query), the route handler checks that the `tenant_id` in the payload exactly matches the `tenant_id` signed into the verified JWT token. If an Acme user attempts to query the Globex handbook, the backend immediately terminates the request with HTTP 403 Forbidden before initiating vector retrieval.

5. Persistent Identity-Keyed Rate Limiting

Most rate limiters track requests by IP address (easily spoofed or shared by multiple users in an office) or by company session (allowing a user to bypass quotas by switching between Acme and Globex). This system implements a **Google Subject-Keyed Quota Engine**:

Quota Window	Limit Threshold	Keying Identifier	Persistence Layer
Daily Window	50 queries / 24 hours	<code>google_user:{google_sub}</code>	SQLite (rate_limits.db)
Weekly Window	200 queries / 7 days	<code>google_user:{google_sub}</code>	SQLite (rate_limits.db)

Storage Implementation: SlowAPI is paired with a custom SQLite storage engine with write-ahead synchronization. Counters persist across backend server restarts and ngrok tunnel refreshes. When an authenticated user makes 30 queries under Acme Corp and 20 under Globex Corporation, their daily quota is exhausted. Further queries trigger an immediate HTTP 429 Too Many Requests with an RFC-compliant `Retry-After` header.

6. Real-Time Quota Monitoring & HTTP 429 Takeover

The frontend synchronizes rate limit status through two complementary components:

- **Usage Modal (GET /limits):** Accessible from the navigation bar. Displays live daily and weekly progress meters, color-coded based on consumption (Emerald <60%, Amber 60%–85%, Rose >85%), with exact remaining query counts and window reset timestamps.
- **HTTP 429 Takeover Screen:** Upon receiving a rate-limit exception, the chat UI triggers a full-screen takeover modal. The modal displays the exhausted tier, remaining cooldown period in seconds, and an animated countdown timer.

7. Edge-Case Hardening & Cost Optimizations

A. Small-Talk Interceptor (Compute & Cost Optimization)

Conversational inquiries (e.g. *'hi'*, *'hello'*, *'good morning'*, *'hey'*) do not require semantic embedding computation or retrieval from company handbooks. An optimized regular expression interceptor catches small-talk in **<1 ms**, returning a polite guidance prompt immediately. This eliminates unnecessary vector database reads and saves valuable LLM context tokens.

B. Character Limit & Client/Server Input Guardrails

To mitigate prompt injection attacks, buffer exploitation, and runaway LLM costs, input queries are strictly validated: the frontend enforces a real-time 500-character counter with visual threshold alerts, while the backend Pydantic schema rejects empty messages, whitespace padding, or queries exceeding 500 characters with HTTP 422 Unprocessable Entity.

C. Production ngrok Header Injection

Free ngrok tunnels present an HTML interstitial warning page on first contact, corrupting automated API requests. All frontend API calls inject ngrok-skip-browser-warning: true across headers, guaranteeing smooth JSON serialization across the tunnel.

8. Automated Testing & Verification Suite

The repository contains an exhaustive automated test suite comprising **121 unit and integration tests** executed via pytest:

Test Module	Scope of Verification
test_isolation.py	Verifies physical collection segregation, cross-tenant rejection, and prompt injection containment.
test_auth_rate_limit_flow.py	Full OAuth lifecycle, bcrypt verification, company switching, and 50/day SQLite limit enforcement.
test_api.py	FastAPI endpoints, Pydantic input validation, CORS headers, and standard HTTP error codes.
test_ingestion.py	PDF extraction fidelity, chunking boundaries, metadata tagging, and ChromaDB vector generation.
test_retriever.py	Semantic similarity search scoring, top-k chunk ranking, and tenant collection boundaries.
test_llm_provider.py	Groq LLaMA 3.3, Gemini, and Claude adapter functionality and fallback error handling.

9. Deployment Topology & Live Evaluation

Cloud Frontend: Hosted on Vercel with automatic global edge delivery: <https://multi-tenant-rag-system.vercel.app>.

Hybrid Private Backend: Hosted on private infrastructure and connected via secure TLS tunnel (ngrok). For live evaluation, please message **WhatsApp: +92 313 5060949** 10 minutes prior to activate the tunnel.

Demo Passwords: Acme Corp: Acme@Admin • Globex Corporation: Globex@Admin (Requires Google OAuth Sign-in).